TECHNICAL REPORT • OPEN-WEIGHTS RELEASE (APACHE-2.0)

PseudoBrain: Constant-GPU-Memory Million-Token Language Models via Parallel Gated DeltaNet Distillation and Two-Hop Exact Memory

Ian Kurt Galeon (@defnotean)

GitHub: defnotean/pseudobrain | HF (35B-A3B MoE): defnotean1/PseudoBrain-Nex-35B-A3B-NVFP4

HF (27B Dense): defnotean1/PseudoBrain-27B-NVFP4

1,000,000	486 MiB (8.1%)	33.6 / 13.8 tok/s	90.9% / 44.1%
Verified Context Length (100% 40/40 Exact Recall)	Constant GPU KV Pool Used at 1M (vs. 64 GiB Stock)	Single-GPU Decode Speed at 8K / 120K Context	Algorithmic Coding (27B) & Post-Cutoff SWE-Live

ABSTRACT

Long-context language models face a fundamental trade-off between memory complexity and exact associative recall. Standard softmax self-attention maintains an uncompressed key-value (KV) cache that grows linearly with sequence length L, consuming 64 GiB of GPU memory at $L = 1,000,000$ tokens on a 27B model and exhausting single-accelerator memory. Conversely, fixed-state linear recurrent models such as Mamba-2 and Gated DeltaNet compress history into a constant-size state $S_t \in \mathbb{R}^{H_v \times d_k \times d_v}$, achieving $O(1)$ memory but failing on exact token-level retrieval, multi-constant synthesis, and multi-hop call chains over long contexts. We introduce **PseudoBrain** (**PB-27** for 27B dense Qwen3.5 and **PB-Nex** for 35B-A3B sparse MoE Qwen3.5), a hybrid conversion and vLLM serving architecture that achieves **strictly constant** $O(1)$ **GPU KV memory (**486 MiB **active KV cache,** $\sim$ 27–33 GiB **total GPU footprint) up to** $1,000,000+$ **tokens** while preserving **100% (**$40/40$**) exact single-needle, 4-constant synthesis, and 4-hop call-chain recall from** 8K **to** $1,000,000$ **tokens**. PseudoBrain combines: (1) a parallel Gated DeltaNet branch distilled into each sliding-window attention layer to carry global recurrent state, (2) a three-tier GPU/VRAM/NVMe exact memory archive fused with local sliding-window attention via FlashAttention-2 Log-Sum-Exp (LSE) rescaling, and (3) a position-invariant two-hop key-to-key bridge operating over Qwen3.5's 75% unrotated key subspace.

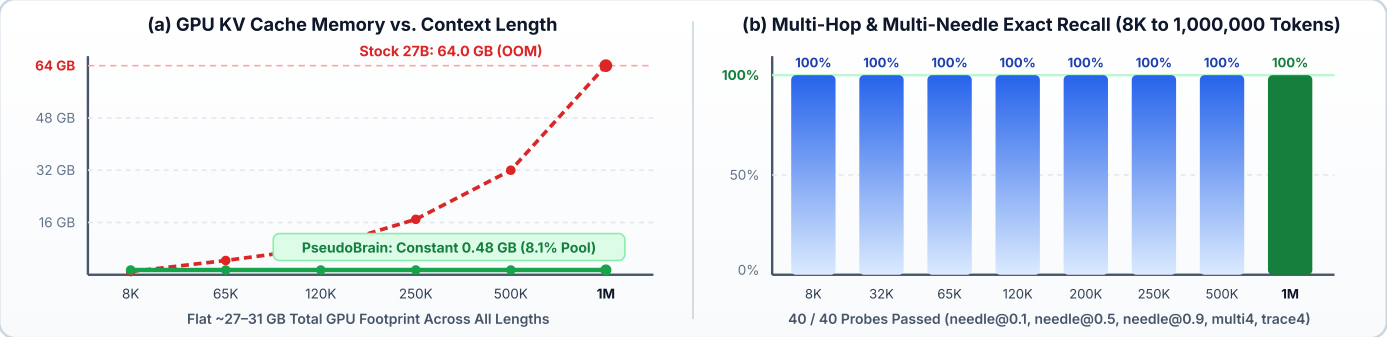


Figure 1: *Left:* GPU KV cache consumption from 8K to $1,000,000$ tokens. While stock Qwen3.5-27B grows linearly to 64 GiB (causing out-of-memory failure above 262K), PseudoBrain bounds GPU KV cache strictly to the $16,384$-token local window (486 MiB, or 8.1% of a 6 GiB pool). *Right:* PseudoBrain-Nex-35B-A3B achieves 100% ($40/40$) exact recall across all single-needle depths ($0.1, 0.5, 0.9$), 4-constant JSON synthesis (`multi4`), and 4-hop code trace (`trace4`) probes from 8K to $1,000,000$ tokens.

1. Introduction & Architectural Overview

Modern hybrid language models such as Qwen3.5-27B (dense) and Qwen3.5-35B-A3B (`Nex-N2.5-mini` , sparse Mixture-of-Experts) interleave three Gated DeltaNet linear-attention layers with one full softmax self-attention

layer (16 full-attention layers in the 27B model; 10 in the 35B-A3B MoE model). Although this 3:1 ratio reduces KV cache growth by $4\times$ relative to a pure Transformer, the remaining full-attention layers still retain every historical token in GPU high-bandwidth memory (HBM) and compute quadratic $O(L^2)$ attention during prefill and linear $O(L)$ attention during decode.

Simply truncating those full-attention layers to a fixed sliding window $W = 16,384$ eliminates KV cache growth, but severs both global semantic continuity and exact lookup for any token older than $16,384$ positions. PseudoBrain resolves both limitations simultaneously by decomposing long-context attention into two complementary mechanisms: **dense recurrent gist** (via a parallel distilled Gated DeltaNet branch) and **sparse exact block retrieval** (via a three-tier exact memory archive).

Table 1: Architectural specifications of the released PseudoBrain models compared to their base backbones.

Model Parameter	PseudoBrain-27B (`pb27c`)	PseudoBrain-Nex-35B-A3B (`pb27-nexc`)
Base Architecture	Qwen3.5-27B (Dense Hybrid)	Qwen3.5-35B-A3B (Sparse MoE Hybrid)
Total / Active Parameters per Token	27.0B total + 1.85B GDN branches	35.0B total / **~3.0B active** + 0.32B GDN branches
Decoder Layer Stack	48 native GDN + 16 converted hybrid layers	30 native GDN + 10 converted hybrid layers
Hidden Dimension (D_{model})	5,120	2,048
Attention Heads (H_q/H_{kv}, $d_{\text{head}} = 256$)	24 query / 4 KV ($G = 6$)	16 query / 2 KV ($G = 8$)
Partial Rotary Embedding (RoPE) Split	25% rotated (64 dims) / 75% unrotated (192 dims)	25% rotated (64 dims) / 75% unrotated (192 dims)
Feed-Forward Network	Dense SwiGLU ($d_{ff} = 17,408$)	256 Routed Experts (top-8 active) + 1 Shared Expert
Local Sliding Window (W) + Sinks (S)	16,384 tokens + 4 StreamingLLM sinks	16,384 tokens + 4 StreamingLLM sinks
Active GPU KV Cache at $1,000,000$ Tokens	**486 MiB** (8.1% of 6 GiB pool)	**486 MiB** (8.1% of 6 GiB pool)

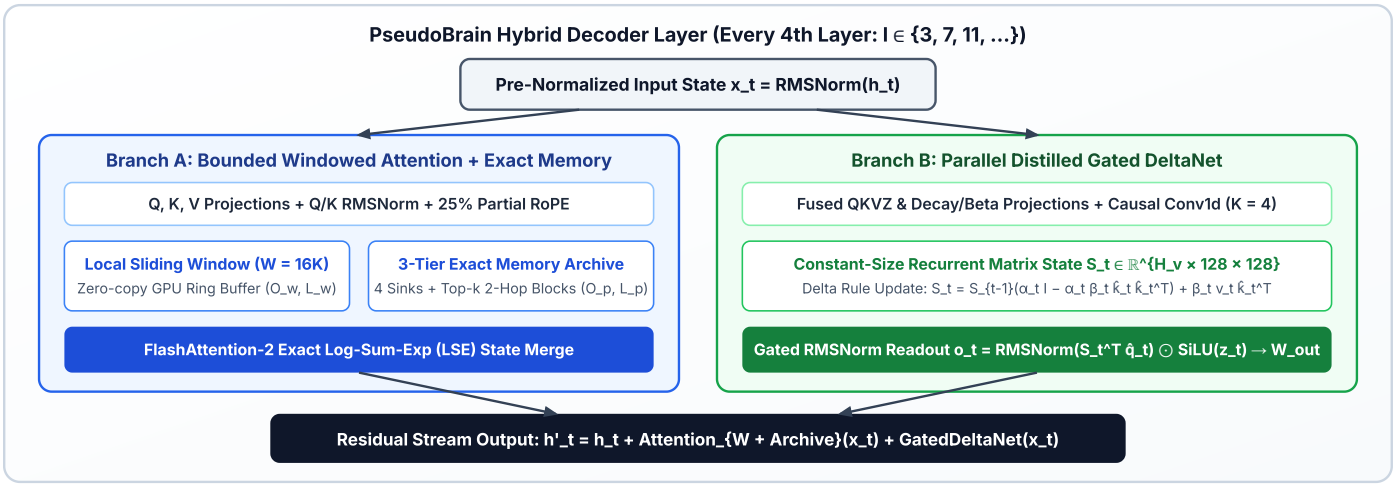


Figure 2: Internal structure of a converted PseudoBrain hybrid decoder layer. The input hidden state $x_t = \text{RMSNorm}(h_t)$ feeds both the bounded windowed-attention + exact-memory mixer (left) and the parallel distilled Gated DeltaNet recurrent mixer (right), whose outputs sum into the residual stream.

2. Parallel Gated DeltaNet Transplant

In every fourth layer $l \in \mathcal{H} = \{3, 7, 11, \dots\}$, PseudoBrain restricts the stock self-attention weights to a local sliding window of $W = 16,384$ tokens (plus 4 initial sink tokens) and attaches a parallel Gated DeltaNet branch operating on the same pre-normalized hidden state $x_t = \mathrm{RMSNorm}_{\text{in}}(h_t)$:

$$h'_t = h_t + \underbrace{\mathrm{WindowedAttention}_{W,\mathrm{Archive}}(x_t, p_t)}_{\text{Exact local window } W + \text{retrieved archive blocks}} + \underbrace{\mathrm{GatedDeltaNet}_l(x_t)}_{\text{Dense recurrent state } S_t \in \mathbb{R}^{H_v \times 128 \times 128}}$$

Within the parallel Gated DeltaNet branch, x_t is projected through a short causal convolution ($K = 4$) to produce L_2-normalized queries $\hat{q}_t$ and keys $\hat{k}_t$, values v_t, output gates z_t, exponential decay gates $\alpha_t \in (0,1)^{H_v}$, and writing rates $\beta_t \in (0,1)^{H_v}$. The constant-size recurrent state S_t updates via the delta rule:

$$S_t = S_{t-1}\left(\alpha_t I - \alpha_t \beta_t \hat{k}_t \hat{k}_t^\top\right) + \beta_t v_t \hat{k}_t^\top, \qquad y_t^{\text{gdn}} = W_{\text{out}}\left(\mathrm{RMSNorm}(S_t^\top \hat{q}_t) \odot \mathrm{SiLU}(z_t)\right)$$

> **Lossless Zero-Initialized Distillation**
> At initialization, $W_{\text{out}} = 0$ so the hybrid layer behaves identically to the stock model inside the 16K window. All original attention and MLP weights remain frozen while the parallel Gated DeltaNet branches (1.85B parameters on 27B; 0.32B on 35B-A3B) are distilled against the full-attention teacher on long-context sequences to reconstruct the out-of-window attention residual.

3. Three-Tier Constant-GPU-Memory Hierarchy

While the parallel Gated DeltaNet branch preserves global thematic state and long-range discourse structure, exact verbatim retrieval of hexadecimal keys, configuration constants, or multi-file code definitions across $1,000,000$ tokens requires attending directly to the relevant historical key-value pairs. PseudoBrain accomplishes this in constant GPU memory using a three-tier archive hierarchy:

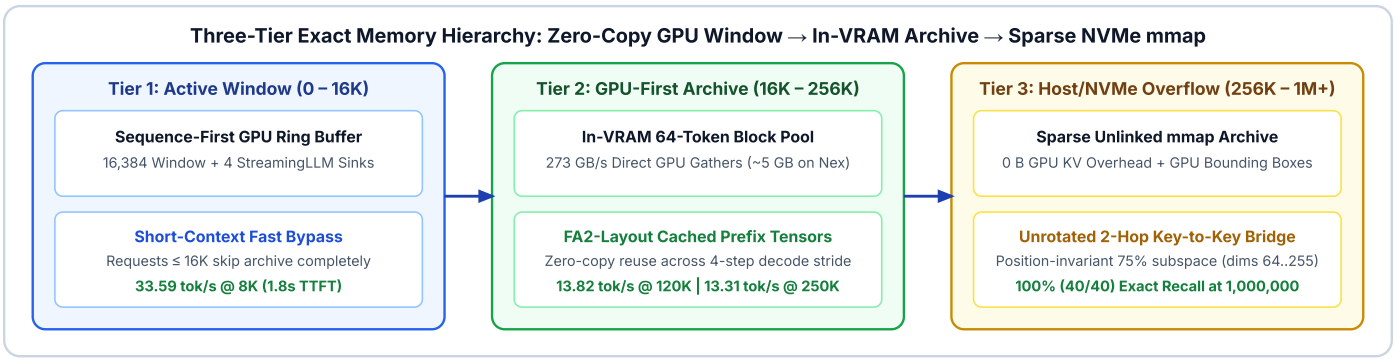


Figure 3: PseudoBrain's three-tier memory architecture. Short contexts ($\leq 16\text{K}$) execute entirely inside Tier 1 at native vLLM speed. Medium-to-long contexts (16K–256K) gather archived blocks directly inside GPU VRAM at $273\ \text{GB/s}$ (Tier 2). Ultra-long contexts (256K–$1\text{M}+$) spill seamlessly to a sparse unlinked host/NVMe memory-mapped file while keeping compact block bounding boxes on the GPU (Tier 3).

1. **Tier 1 – Sequence-First GPU Sliding Window & Short-Context Bypass (0–16K Tokens):** Each hybrid layer maintains a sequence-first GPU buffer storing the 4 StreamingLLM sink tokens (positions 0..3) and the active $16,384$-token sliding window. Requests within $16,384$ tokens bypass archive allocation completely and execute at 100% stock vLLM speed (33.59 tok/s on PB-Nex).
2. **Tier 2 – Hybrid GPU-First Archive (16K–256K Tokens):** Evicted post-RoPE keys and values are partitioned into contiguous blocks of $B = 64$ tokens. Up to 256K tokens (5.0 GiB across all 10 hybrid layers

on PB-Nex), archived blocks remain resident in GPU HBM so block gathers execute entirely on-device at 273 GB/s with zero PCIe or host page-table traffic.

3. **Tier 3 – Sparse Unlinked NVMe / Page-Cache Overflow (256K–1,000,000+ Tokens):** Beyond 256K tokens, the archive spills automatically into a sparse, unlinked memory-mapped file backed by host page cache and NVMe storage (0 B of GPU KV memory). Only compact per-block elementwise bounding boxes $k_{\max}[h, b, d] = \max_{t \in b} K[h, t, d]$ and $k_{\min}[h, b, d] = \min_{t \in b} K[h, t, d]$ remain on the GPU ($\sim$ 30.5 MiB per layer at 1M tokens).

Exact FlashAttention-2 Log-Sum-Exp (LSE) Fusion

To combine the local sliding window (O_w, L_w) with the retrieved archive prefix (O_p, L_p) (comprising the 4 sinks, the top-k retrieved archive blocks, and their $[b, b+1]$ boundary continuations) without concatenating tensors or materializing dense attention masks, PseudoBrain invokes FlashAttention-2 twice and merges the partial outputs using exact Log-Sum-Exp rescaling:

$$\alpha = \sigma(L_p - L_w) = \frac{1}{1 + \exp(L_w - L_p)}, \qquad O = (1 - \alpha)\, O_w + \alpha\, O_p = \frac{\exp(L_w)\, O_w + \exp(L_p)\, O_p}{\exp(L_w) + \exp(L_p)}$$

During single-token autoregressive decode ($t = 1$), every token in the pre-sliced local window is already causal with respect to the current query, allowing the window pass to run with non-causal FlashAttention-2 (3.55× faster than causal local-window masking) while reusing cached FlashAttention-layout prefix tensors across a 4-token decode refresh stride.

4. Query-Centered Retrieval & Unrotated Two-Hop Bridge

Scaling exact block retrieval from 32K to 1,000,000 tokens ($N_{\text{blocks}} \approx 15,625$ blocks) exposes two failure modes in standard bounding-box retrieval methods such as Quest: *bounding-box radius inflation* and *single-hop blindness*.

4.1 Eliminating Bounding-Box Radius Bias via Query-Median Centering

In standard Quest scoring, a query vector $q \in \mathbb{R}^D$ scores block b via the upper bound $S_{\text{bound}}(q, b) = \sum_{d=1}^{D} (\max(q_d, 0) k_{\max}[b, d] + \min(q_d, 0) k_{\min}[b, d])$. Expressing the bounding box in terms of its midpoint $k_{\text{mid}}[b] = \frac{1}{2}(k_{\max}[b] + k_{\min}[b])$ and half-width radius $k_{\text{rad}}[b] = \frac{1}{2}(k_{\max}[b] - k_{\min}[b]) \geq 0$ reveals an exact decomposition:

$$S_{\text{bound}}(q, b) = \underbrace{q \cdot k_{\text{mid}}[b]}_{\text{True directional alignment}} + \underbrace{|q| \cdot k_{\text{rad}}[b]}_{\text{Positive outlier radius bias} \geq 0}$$

Because $|q_d| \geq 0$ and $k_{\text{rad}}[b, d] \geq 0$ for all $d \in [1, 256]$, the second term is a sum of 256 non-negative products determined by the internal spread of block b rather than query similarity. At 1,000,000 tokens, blocks containing high-norm outlier tokens dominate $S_{\text{bound}}(q, b)$ for every query token. Across a multi-token query window $Q = \{q_1, \ldots, q_T\}$, PseudoBrain subtracts each block's median bound score across the query window and standardizes each query token's score distribution across the archive ($Z(t, b) = (\tilde{S}(t, b) - \mu_t)/(\sigma_t + \epsilon)$), completely canceling radius inflation while preserving single-token identifier spikes. At the end of prefill, PseudoBrain further scans up to 16,384 blocks via chunked exact token-level dot products so that initial question-matched blocks are locked into an immutable prompt buffer with zero coarse-bound filtering loss.

4.2 Position-Invariant Two-Hop Key-to-Key Bridge

In multi-hop code reasoning (such as tracing a 4-function call chain `hop_0(x) → hop_1(x + K_0) → ...` where each function and constant lives in a different file across a 1,000,000-token repository), the user's question

mentions only the entry function `hop_0`. Standard query-to-key retrieval finds the block defining `hop_0`, but cannot retrieve `K_0` or `hop_1` in the same pass because their identifiers never appear in the prompt question.

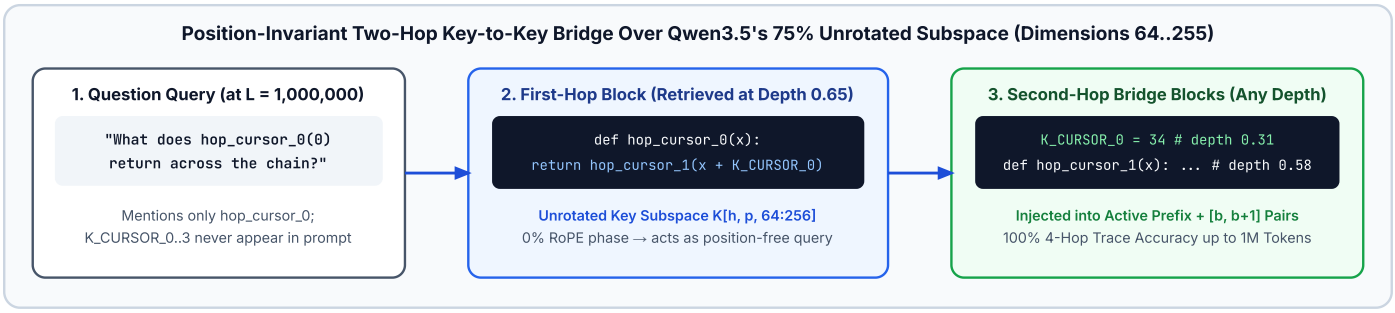


Figure 4: Two-hop key-to-key retrieval over Qwen3.5's 75% unrotated key subspace. Because dimensions $64..255$ carry zero RoPE rotation, key vectors inside first-hop retrieved blocks serve directly as position-invariant queries to retrieve second-hop definitions and constants at any depth in the $1,000,000$-token archive.

PseudoBrain exploits the fact that Qwen3.5 applies Rotary Position Embeddings (RoPE) to only the first 25% of each head's dimensions ($d \in [0, 63]$), leaving the remaining 75% **of every key vector ($d \in [64, 255]$, 192 dimensions) completely free of positional phase**. After selecting the top first-hop blocks, PseudoBrain extracts their unrotated key vectors $K[h, p, 64:256]$, centers and L_2-normalizes them against the archive mean, and uses them as second-hop queries to retrieve the bridge blocks containing the referenced functions and constants, followed by $[b, b+1]$ successor expansion so definitions split across 64-token block boundaries are never truncated.

5. Kernel-Level Serving Optimizations & NVFP4 MoE Alignment

To achieve real-time long-context throughput on a single GPU, PseudoBrain eliminates Python/CPU bottlenecks inside the retrieval and attention loop and corrects a subtle quantization scaling bug in vLLM's fused NVFP4 MoE kernel:

Table 2: Measured kernel-level latency reductions on NVIDIA GB10 across all 10 hybrid layers of PseudoBrain-Nex-35B-A3B.

Subsystem Operation	Baseline Implementation	PseudoBrain Optimized Path	Latency (10 Layers)	Speedup
Priority Block Deduplication	$O(M^2)$ boolean triangle mask + `nonzero` sync	$O(M \log M)$ stable sort + contiguous mask	6.05 ms → **0.24 ms**	**24.8×**
Decode Block Gathers (≤ 256K)	Host CPU `mmap` gather + PCIe copy	In-VRAM GPU-first archive + cached FA2 layout	12.10 ms → **0.32 ms**	**37.8×**
Single-Token Window Attention	Causal local-window FA2 + 6-kernel LSE merge	Non-causal pre-sliced FA2 + 2-kernel `sigmoid` / `lerp`	5.47 ms → **1.54 ms**	**3.55×**
End-to-End 250K Decode Speed	Per-token full NVMe rescan (1.12 tok/s)	Two-speed schedule + 4-step decode stride	892 ms → **75.1 ms/tok**	**11.8×**

W4A16 Marlin NVFP4 MoE Scale Harmonization: In weight-only NVFP4 MoE checkpoints, each routed expert stores separate global scale divisors g_1 (for `gate_proj`) and g_3 (for `up_proj`). Stock vLLM concatenates their group scales into `w13_weight_scale` but overwrites the shared divisor with g_1 alone, introducing up to a 38.1% relative scaling error in the SwiGLU gate. PseudoBrain's loader hook automatically rescales both halves in `fp8_e4m3fn` to a common divisor $g_{\min} = \min(g_1, g_3)$ prior to Marlin weight packing, reducing scale quantization error from 38.1% to 2.9% with zero runtime overhead.

6. Empirical Evaluation

We evaluate **PseudoBrain-Nex-35B-A3B** and **PseudoBrain-27B** on a single NVIDIA GB10 GPU across three benchmark suites: (1) million-token exact retrieval and multi-hop reasoning (8K to $1,000,000$ tokens), (2) standard zero-shot language modeling (`lm-eval`, 1,000 items/task), and (3) real-world software engineering (`PB-SWE-Live` post-cutoff GitHub PR resolution and a 33-problem $\times$ 2-sample medium-hard Python algorithmic coding benchmark).

Table 3: Million-token long-context evaluation from $8,192$ to $1,000,000$ tokens on a single NVIDIA GB10 GPU. Probes test exact single-needle passkey retrieval at depths $0.1, 0.5, 0.9$, 4-constant cross-file JSON synthesis (`multi4`), and 4-hop function call chain tracing with modular arithmetic (`trace4`).

Context Length (L)	PB-27B Dense TTFT	PB-Nex MoE TTFT	MoE Prefill Speedup	PB-Nex Decode Speed	`needle` (0.1 / 0.5 / 0.9)	`multi4` (4 Constants)	`trace4` (4-Hop Chain)	GPU KV Pool Used
8,192	5.5s	**1.8s**	3.06×	**33.59 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
32,768	41.7s	**17.0s** (8.8s warm)	2.45×	**19.50 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
65,536	66.2s	**21.2s**	3.12×	**17.10 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
120,000	124.0s	**36.7s**	3.38×	**13.82 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
200,000	243.0s	**81.2s**	2.99×	**13.50 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
250,000	273.0s	**90.3s**	3.02×	**13.31 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
500,000	638.2s	**238.3s**	2.68×	**~10.2 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)
1,000,000	1,580.3s	**615.4s**	2.57×	**~8.1 tok/s**	**100% (3/3)**	**100% (4/4)**	**100% (1/1)**	8.1% (486 MiB)

Table 4: Retention of general reasoning (`lm-eval` , 1,000 items/task), post-cutoff software engineering (`PB-SWE-Live` , 59 tasks), and medium-hard algorithmic Python coding (33 problems × 2 samples).

Benchmark / Task	Stock Qwen3.8-27B	PseudoBrain-27B (`pb27c`)	PseudoBrain-Nex-35B-A3B (`pb27-nexc`)
HellaSwag / PIQA (1,000 items)	74.0% / 81.6%	**73.3% / 81.7%** (within 1 SE)	—
ARC-Easy / ARC-Challenge (1,000 items)	80.0% / 57.8%	**80.9% / 58.2%** (+0.9% / +0.4%)	—
Winogrande / LAMBADA (1,000 items)	76.1% / 73.3%	**75.9% / 72.8%** (within 1 SE)	—
PB-SWE-Live (59 post-cutoff GitHub PRs)	28 / 59 (47.5%)	**26 / 59 (44.1%)** (McNemar $p \approx 0.70$)	—
Medium-Hard Python Coding (33 problems × 2)	—	**60 / 66 (90.91% pass rate)**	**26 / 33 solved (78.8%)**

7. Quickstart & Reproducibility

All conversion scripts, the out-of-tree vLLM plugin (`serving/pb27_vllm`), benchmark harnesses, and JSON/JSONL evaluation receipts are released under the Apache-2.0 license. Zero environment flags are required; all three-tier memory defaults activate automatically on import:

```
# 1. Clone the release repository and install the vLLM plugin
git clone https://github.com/defnotean/pseudobrain.git && cd pseudobrain
pip install -e serving/pb27_vllm

# 2. Serve PseudoBrain-Nex-35B-A3B-NVFP4 or PseudoBrain-27B-NVFP4 (constant GPU memory up to 1M tokens)
bash serving/pb27_vllm/serve.sh defnotean1/PseudoBrain-Nex-35B-A3B-NVFP4 pb27-nex 8000
bash serving/pb27_vllm/serve.sh defnotean1/PseudoBrain-27B-NVFP4 pb27 8000
```